

ITE351-LOCUS

| AI-Driven Household Context-Awareness for Predictive Cleaning

Table of Contents

1. [Introduction](#)
2. [Datasets](#)
3. [Methodology](#)
4. [Evaluation & Analysis](#)
5. [Related Work](#)
6. [Conclusion](#)
7. [References](#)

Team Members

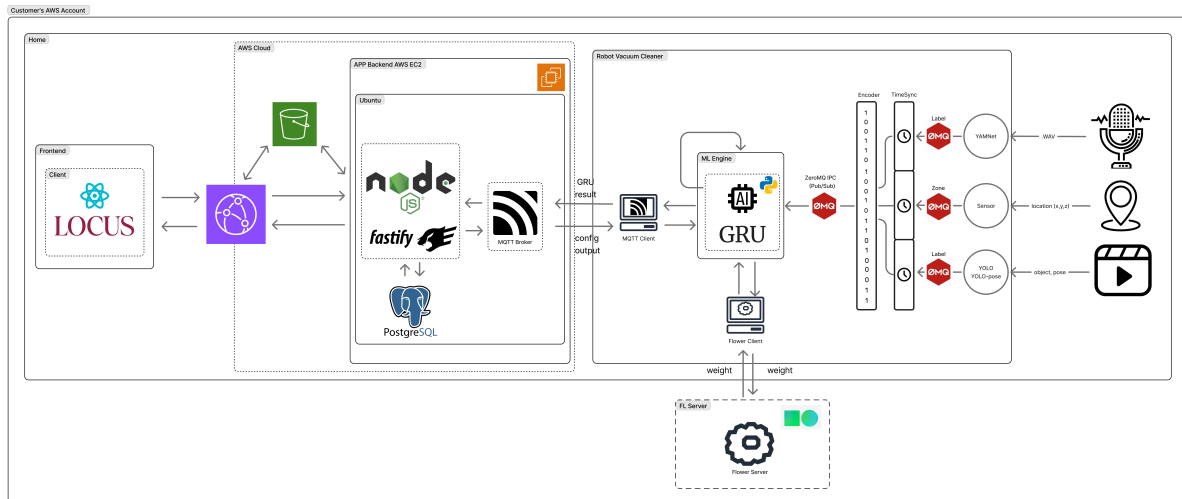
Name	Organization	Email
Hanyeong Go	Department of Information Systems, Hanyang University	lilla9907@hanyang.ac.kr
Junhyung Kim	Department of Information Systems, Hanyang University	combe4259@hanyang.ac.kr
Dayeon Lee	Department of Sports Science, Hanyang University	ldy21@hanyang.ac.kr
Seunghwan Lee	Department of Information Systems, Hanyang University	shlee5820@hanyang.ac.kr

Video Presentation

Video Link: <https://bit.ly/4rW1g8E>

https://www.youtube.com/watch?v=iMFx_97CZnE

Architecture



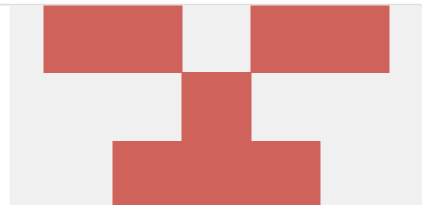
Links

Github:

SEAIG10

SEAIG10 has 6 repositories available. Follow their code on GitHub.

 <https://github.com/SEAIG10>



I. Introduction

Project Overview

Current robotic vacuum cleaners operate in a **reactive** manner—they simply follow user commands or preset schedules without understanding the household context. This project transforms the robot vacuum from a passive tool into an **intelligent household partner** that proactively understands and responds to real-time home environments.

Our system enables the robot to perceive the household through multiple modalities:

Modality	Sensor/Model	What it captures
Objects & People	YOLOv11n	Furniture, appliances, human presence
Human Pose	YOLOv11n-pose	Body posture (sitting, standing, cooking)

Modality	Sensor/Model	What it captures
Sounds	YAMNet	Household activities (cooking, dishwashing, TV)
Location	Spatial/Zone tracking	Semantic room labels (kitchen, bedroom)
Time	Temporal features	Time of day, day of week patterns

These multi-modal inputs are fused by an **AttentionContextEncoder** into a unified context vector, which is then fed into a **GRU model** to predict contamination probability for each zone. The result: instead of waiting for user commands, the robot **anticipates when and where cleaning will be needed** based on household activity patterns.

Furthermore, we employ **Federated Learning with Personalization (FedPer)** to ensure both privacy protection and continuous model improvement:

- **Base Layer (Shared):** Common household patterns (e.g., "cooking sounds → kitchen contamination") are learned collaboratively across multiple households via federated learning, without sharing raw sensor data.
- **Head Layer (Personal):** Household-specific habits (e.g., "this family cooks at 7 PM") are trained on-device and never uploaded to the server.

This architecture allows the robot to become smarter over time—benefiting from collective learning while preserving individual privacy.

Motivation: Why are we doing this?

This project addresses three key limitations in current robotic vacuum cleaners:

1. **Reactive Operation:** Despite hardware advances, current commercial robot vacuums only operate on preset schedules or direct commands, lacking the ability to anticipate contamination based on household context.
2. **Privacy Concerns:** Conventional systems transmit sensitive video/audio data to cloud servers for AI processing, raising significant privacy risks for household environments.
3. **Lack of Personalization:** Generic cleaning patterns ignore household-specific living habits (cooking times, pet activity, guest visits), reducing cleaning efficiency and user satisfaction.

What do we want to see at the end?

Goal	Description	Key Technology
Predictive Cleaning	Anticipate contamination timing and location by fusing multi-modal context	YOLOv11 + YAMNet + GRU
Privacy Assurance	Process all sensitive data on-device without cloud transmission	Edge AI + Federated Learning
Personalization	Adapt to household-specific patterns and user preferences	FedPer Architecture

II. Datasets

1. YOLO Datasets (Visual Feature Extraction)

- Objective: Household object recognition and contamination detection for the visual perception of robotic vacuums.
- Final Data: HomeObjects-3K + HD10K

- HomeObjects-3K



Dataset	Purpose	Total	Train / Val	Source
HomeObjects-3K	Household Object Recognition	2,689	2,285 / 404	Ultralytics, License: AGPL-3.0
HD10K	Contamination Detection	10,000	8,000 / 2,000	IROS 2022 Dataset
Final Data	YOLO Training	12,689	10,285 / 2,404	

- Classes(14): bed, sofa, chair, table, lamp, tv, laptop, wardrobe, window, door, potted_plant, photo_frame, solid_waste, liquid_stain
- YOLO Training Settings:
 - Base Model: YOLOv11n (COCO pretrained)
 - Freeze: 10 layers
 - Learning Rate: 0.001
 - Epochs: 50 (Early Stopping → 24)

- Image Size: 640×640, Batch Size: 4

2. YAMNet Datasets (Auditory Feature Extraction)

- Objective: Context awareness based on household environmental sounds for robotic vacuums (Auditory).
- Source: Google AudioSet → Refined Home Audio Dataset constructed after background/noise removal.
- Final Data: Total 22,212 clips (Train 17,770 / Val 2,221 / Test 2,221)
 - Classes(17): speech, door, dishes, cutlery, chopping, frying, microwave, blender, water_tap, sink, toilet_flush, telephone, chewing, television, footsteps, vacuum, hair_dryer
 - Input Shape: (N, 1024)
- YAMNet Training Settings:
 - Backbone: YAMNet (MobileNetV1-based, 1024-dim, Frozen)
 - Head: Dense(512-ReLU) → Dropout(0.2) → Dense(17-Sigmoid)
 - Loss: Binary Cross Entropy (Weighted for class imbalance)
 - Epochs: 100 (Early Stopping)
 - Final Model: TFLite FP16, 17MB

3. GRU Datasets (Context-Based Cleaning Prediction)

- Objective: Prediction of zones requiring cleaning based on context.
- **Data Generation:** Synthetic dataset generated using realistic household behavior simulation
 - Simulation Period: 2,000 days × 3 random seeds for diversity
 - Sampling Rate: 15-minute intervals (4 samples/hour)
 - Total Timesteps: ~220,000
- Input Features (per timestep):

Modality	Dimensions	Description
Time	10	Hour, day of week, etc.
Spatial	4	Zone one-hot encoding
Visual	14	YOLO object detection
Audio	17	YAMNet sound classification
Pose	51	Human keypoints (17 × 3)
Total	96	→ 160-dim via AttentionContextEncoder

- Sequence Length: 30 timesteps per sample
- Output: Contamination probability for 4 zones (balcony, bedroom, kitchen, living_room)
- Label Generation: Binary label based on pollution increase (current > previous → 1, else → 0)
- Train/Val Split: 80% / 20%

III. Methodology

1. Choice of Algorithms & Models

1. Visual Context: YOLOv11n-pose (Quantized)

- We selected YOLOv11n, the most lightweight version of the YOLO model family. Specifically, we adopted the Pose version to go beyond simple object detection and utilize Keypoints to precisely analyze human actions (e.g., 'sitting', 'standing').
- Optimization: We applied TensorFlow Lite Quantization for deployment on edge device environments. This effectively reduced the model size while maintaining inference speed.

2. Audio Context (Transfer learning, Quantized)

- YAMNet is designed based on the MobileNetV1 architecture, which is optimized for mobile devices. It offers the advantage of efficiently classifying household environmental sounds (such as dishwashing) without the high computational costs associated with large audio transformer models.

3. Sequential Prediction: GRU (Gated Recurrent Unit)

- We selected GRU over LSTM for time-series prediction. GRU has a simpler gate structure than LSTM, requiring less computation, making it more suitable for edge devices. Simultaneously, it effectively captures the Temporal Dependencies of household living patterns.

4. Multi-modal Fusion: AttentionContextEncoder

- To fuse heterogeneous sensor data (visual, audio, pose, spatial, temporal) into a unified representation, we designed an AttentionContextEncoder using Multi-Head Attention mechanism. Each modality is first projected to a common 64-dimensional embedding space, then cross-modal attention allows each modality to attend to all others, learning complex patterns like "sofa + TV + sitting = living room relaxation".

```
# AttentionContextEncoder: Multi-modal fusion with attention mechanism
class AttentionContextEncoder(tf.keras.Model):
    def __init__(self, output_dim=160):
        super().__init__()
        # Project each modality to common embedding space (64-dim)
        self.visual_proj = Dense(64, activation='relu') # YOLO: 14 → 64
        self.audio_proj = Dense(64, activation='relu') # YAMNet: 17 → 64
        self.pose_proj = Dense(64, activation='relu') # Pose: 51 → 64
        self.spatial_proj = Dense(64, activation='relu') # Zone: 4 → 64
        self.time_proj = Dense(64, activation='relu') # Time: 10 → 64

        # Cross-modal attention: each modality attends to all others
        self.attention = MultiHeadAttention(num_heads=4, key_dim=64)
        self.layer_norm = LayerNormalization()

        # Fusion layer: concatenate attended features → 160-dim context
        self.fusion = Dense(output_dim, activation='relu')
```

```

def call(self, inputs):
    # inputs: dict with keys ['visual', 'audio', 'pose', 'spatial', 'time']
    embeddings = tf.stack([
        self.visual_proj(inputs['visual']),
        self.audio_proj(inputs['audio']),
        self.pose_proj(inputs['pose']),
        self.spatial_proj(inputs['spatial']),
        self.time_proj(inputs['time'])
    ], axis=1) # Shape: (batch, 5, 64)

    # Self-attention across modalities
    attended = self.attention(embeddings, embeddings)
    attended = self.layer_norm(attended + embeddings) # Residual

    # Flatten and fuse to 160-dim context vector
    return self.fusion(tf.reshape(attended, [-1, 5 * 64]))

```

5. Data Synchronization: TimeSyncBuffer (Custom Algorithm)

- Standard synchronization is challenging because vision sensors (~30fps) and audio sensors (~1s) operate at different frequencies. We implemented a custom synchronization algorithm inspired by **ROS ApproximateTimeSynchronizer** that aligns asynchronous data streams within a tolerance window (SLOP) to merge them into a single context vector.

```

# TimeSyncBuffer: ROS ApproximateTimeSynchronizer-inspired synchronization
QUEUE_SIZE = 10 # Max messages per sensor queue
SLOP = 0.5 # Tolerance window (seconds)

class TimeSyncBuffer:
    def __init__(self):
        self.sensor_queues = {
            'visual': deque(maxlen=QUEUE_SIZE),
            'audio': deque(maxlen=QUEUE_SIZE),
            'pose': deque(maxlen=QUEUE_SIZE),
            'spatial': deque(maxlen=QUEUE_SIZE),
            'time': deque(maxlen=QUEUE_SIZE)
        }
        self.context_buffer = deque(maxlen=30) # 30 timesteps for GRU

    def try_synchronize(self):
        """Align multi-modal sensor data within SLOP tolerance"""
        if not all(len(q) > 0 for q in self.sensor_queues.values()):
            return None

        # 1. Find pivot: latest timestamp among first messages
        pivot_ts = max(q[0]['timestamp'] for q in self.sensor_queues.values())

        # 2. Find closest message to pivot in each queue

```

```

candidates = {}
for name, queue in self.sensor_queues.items():
    closest = min(queue, key=lambda m: abs(m['timestamp'] - pivot_ts))
    candidates[name] = closest

# 3. Verify all within SLOP tolerance
timestamps = [c['timestamp'] for c in candidates.values()]
if max(timestamps) - min(timestamps) <= SLOP:
    return self._fuse_context(candidates)

# 4. Discard oldest message and retry
oldest_queue = min(self.sensor_queues.keys(),
                    key=lambda k: self.sensor_queues[k][0]['timestamp'])
self.sensor_queues[oldest_queue].popleft()
return None

```

6. Federated Learning: FedPer Architecture

- We adopted the FedPer architecture to balance privacy protection and model performance. Unlike standard FedAvg, this approach separates the model into a Base Layer (Shared) and a Head Layer (Personal), allowing the robot to learn unique household habits without sharing sensitive label distribution information.

```

# FedPer Architecture: Base (shared) + Head (personal) separation
class FedPerGRUModel:
    def __init__(self, input_shape=(30, 160), num_zones=4):
        self.base_model = self._build_base_model(input_shape) # Shared
        self.head_model = self._build_head_model(num_zones)    # Personal

    def _build_base_model(self, input_shape):
        """Base Layer: Shared across all households via FL server"""
        inputs = Input(shape=input_shape)
        x = GRU(64, return_sequences=True)(inputs)
        x = GRU(32, return_sequences=False)(x) # Output: (batch, 32)
        return Model(inputs, x, name="base_model") # ~42.8K params

    def _build_head_model(self, num_zones):
        """Head Layer: Personal, never shared with server"""
        inputs = Input(shape=(32,))
        x = Dense(16, activation='relu')(inputs)
        x = Dropout(0.3)(x)
        outputs = Dense(num_zones, activation='sigmoid')(x) # ~0.6K params
        return Model(inputs, outputs, name="head_model")

    def federated_sync(self, server_weights):
        """Apply global base weights, keep head local"""
        self.base_model.set_weights(server_weights)
        # head_model weights are NEVER uploaded to server

```

7. FL Server Aggregation: FedAvg with Flower

- The central server collects base layer updates from multiple households and aggregates them using the FedAvg algorithm. We implemented this using the Flower framework, which provides gRPC-based communication and built-in aggregation strategies.

```
# FL Server: Aggregates base layer weights from multiple clients
class LocusFedAvg(fl.server.strategy.FedAvg):
    def __init__(self, initial_weights_path, ckpt_dir, **kwargs):
        # Load initial global weights
        self.global_weights = np.load(initial_weights_path, allow_pickle=True)
        self.ckpt_dir = Path(ckpt_dir)
        super().__init__(initial_parameters=ndarrays_to_parameters(self.global_weights), **kwargs)

    def aggregate_fit(self, server_round, results, failures):
        """Aggregate base layer updates from all participating clients"""
        # 1. Collect updates from each client
        for client, fit_res in results:
            log_fl_event("server", "update_received",
                        client_id=client.cid, round=server_round)

        # 2. Perform FedAvg: weighted average by number of samples
        aggregated = super().aggregate_fit(server_round, results, failures)
        parameters, metrics = aggregated

        # 3. Save aggregated global weights as checkpoint
        if parameters is not None:
            weights = parameters_to_ndarrays(parameters)
            ckpt_path = self.ckpt_dir / f"round_{server_round}.npz"
            np.save(ckpt_path, np.array(weights, dtype=object))

        return aggregated
```

IV. Evaluation & Analysis

1. YOLO Object Detection Model Results

- Validation Metrics

Metric	Score
Precision	0.637
Recall	0.544
mAP50	0.566
mAP50-95	0.389

- Loss Values

Loss	Values
Box Loss	0.988
Class Loss	0.793
DFL Loss	0.954

⇒ Visual object recognition performance is at a moderate level.

2. YAMNet Model Results

- Overall Performance Summary

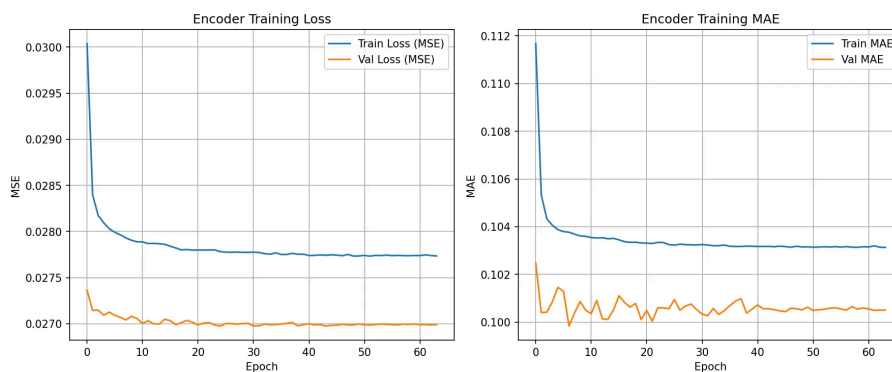
Item	Value
Total Samples	22,212
Macro AUC	0.9899
Macro AUPR	0.9705

- Per-Class Performance(Top5)

Rank	Class	AUC	AUPR
1	vacuum	0.9980	0.9957
2	hair_dryer	0.9981	0.9941
3	toilet_flush	0.9962	0.9911
4	blender	0.9948	0.9886
5	telephone	0.9954	0.9870

⇒ Accurately classifies auditory events.

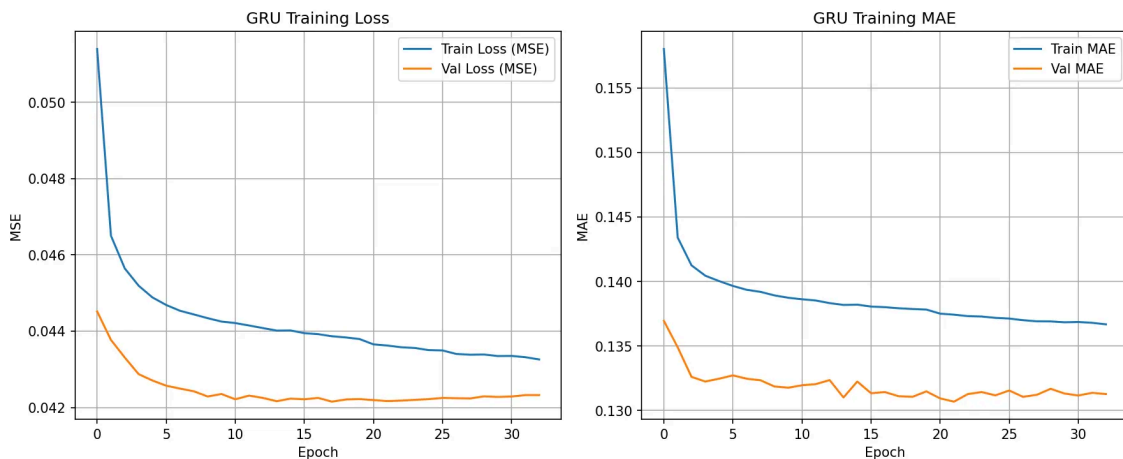
3. AttentionContextEncoder Model Results



- Training Epochs: 64 (Best Epoch: 44)
- Loss Convergence: Train 0.0300 → 0.0277, Val 0.0274 → 0.0270
- MAE Performance: Train 0.1117 → 0.1031, Val 0.1025 → 0.1005
- Best Performance: Val Loss 0.0270, Val MAE 0.1005 (Epoch 44)
- Stable convergence with minimal overfitting gap

⇒ The AttentionContextEncoder effectively fuses multi-modal sensor data (YOLO, YAMNet, Pose, spatial, Time) into 160-dimensional context vectors with low prediction error.

4. GRU Model Results (FedPer Head Layer)



- Training Epochs: 33 (Best Epoch: 18)
- Loss Convergence: Train 0.0514 → 0.0433, Val 0.0445 → 0.0423
- MAE Performance: Train 0.1580 → 0.1367, Val 0.1369 → 0.1313
- Best Performance: Val Loss 0.0422, Val MAE 0.1311 (Epoch 18)
- Consistent train-val loss gap indicates good generalization

⇒ The GRU-based sequential model successfully predicts zone-specific pollution probabilities from 30-timestep context sequences, achieving Mean Absolute Error of 0.131 on validation data.

5. Why GRU? Architectural Rationale

Edge Device Constraints

Our target deployment environment (robot vacuum embedded system) imposes strict constraints:

Constraint	Requirement	Reason
Latency	< 100ms per inference	Real-time response to context changes
Memory	< 1MB model size	Limited RAM on embedded processors
Power	Minimize computation	Battery-powered operation (2-3 hours)
Concurrent Models	YOLO + YAMNet + GRU	Must run simultaneously

Architectural Comparison

Model	Gate Structure	Complexity	Edge Suitability
GRU	2 gates (reset, update)	$O(n)$	Optimal
LSTM	3 gates (forget, input, output)	$O(n)$	Moderate
Transformer	Self-attention	$O(n^2)$	Too Heavy

Why GRU over LSTM?

Based on architectural analysis and literature on smart home time-series prediction:

1. **Simpler Gate Structure:** GRU uses 2 gates while LSTM uses 3 gates, resulting in fewer parameters and faster computation for equivalent hidden dimensions.
2. **Comparable Performance:** Chung et al. [1] demonstrated that GRU achieves similar accuracy to LSTM on sequence modeling tasks. Recent smart home energy prediction studies [2][3] confirm this finding in household time-series applications.
3. **Edge Optimization:** GRU's reduced complexity makes it more suitable for on-device deployment where multiple models must run concurrently.

Why not Transformer?

1. **Quadratic Complexity:** Self-attention has $O(n^2)$ complexity with sequence length, problematic for our 30-timestep sequences when running alongside YOLO and YAMNet.
2. **Memory Overhead:** Transformer architectures require significantly more parameters and runtime memory, exceeding typical embedded system constraints.
3. **Overkill for Task:** For household pattern prediction with relatively short sequences, the global attention mechanism provides diminishing returns compared to recurrent approaches.

Conclusion: GRU provides the optimal balance between model capacity and computational efficiency for edge deployment. Its simpler architecture enables real-time inference on resource-constrained robot vacuum hardware while maintaining sufficient capacity to learn household temporal patterns.

V. Related Work

This project builds upon several foundational AI technologies and research advances in computer vision, audio processing, federated learning, and multi-modal sensor fusion for robotics applications.

1. Object Detection and Pose Estimation

YOLOv11 for Robotic Vision: YOLOv11, developed by Ultralytics and unveiled at YOLO Vision 2024 conference [4], represents the state-of-the-art in real-time object detection. Our project leverages YOLOv11n-pose, which supports 17-keypoint human pose estimation trained on the COCO keypoints dataset. YOLOv11 achieves 25-40% latency reduction compared to YOLOv10 while maintaining higher accuracy with 22% fewer parameters, making it ideal for edge deployment in robotic applications. The pose estimation capability enables robots to interpret physical actions in real-time, which is critical for understanding household activities and predicting contamination patterns.

Deployment Optimization: Ultralytics provides comprehensive documentation for deploying YOLO models on edge devices through TensorFlow Lite quantization, enabling real-time inference on resource-constrained platforms such as robot vacuums and mobile devices.

2. Audio Event Classification

YAMNet and AudioSet: YAMNet is a pretrained deep neural network developed by Google that predicts 521 audio event classes based on the AudioSet-YouTube corpus [5]. Built on the MobileNet_v1 depthwise-separable convolution architecture, YAMNet employs 3.7M weights and performs 69.2M multiplies for each 960ms input frame at 16kHz sampling rate. The model was

trained on 1,574,587 ten-second YouTube excerpts from AudioSet's unbalanced train segments [6], covering diverse environmental sounds including household activities.

Transfer Learning Capabilities: TensorFlow Hub provides YAMNet as a reusable module for transfer learning on custom audio classification tasks. Our project fine-tunes YAMNet on household-specific sounds (17 classes including cooking, dishwashing, vacuum operation) to achieve domain-specific performance superior to the generic pretrained model.

3. Federated Learning with Personalization

FedPer Architecture: Arivazhagan et al. [7] introduced Federated Learning with Personalization Layers (FedPer) in their seminal work "Federated Learning with Personalization Layers". FedPer addresses statistical heterogeneity in federated learning by separating the model into a shared base layer (feature extractor trained collaboratively across clients) and personalized head layers (trained locally and never shared). This architecture enables each client to adapt to local data distributions while benefiting from collective learning.

Privacy-Preserving Personalization: Recent extensions of FedPer, such as FedPerGNN for graph neural networks, demonstrate 4.0-9.6% lower prediction errors compared to state-of-the-art privacy-preserving methods under satisfactory privacy budgets. The base + personalization layer paradigm effectively combats the adverse effects of statistical heterogeneity in federated networks, which is critical for household-specific cleaning pattern learning in our system.

4. Recurrent Neural Networks for Time Series Prediction

GRU Architecture: Chung et al. [1] introduced the Gated Recurrent Unit in "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling". Their experiments demonstrated that GRU achieves comparable performance to LSTM on sequence modeling tasks while using fewer parameters due to its simpler 2-gate structure (reset and update gates) compared to LSTM's 3-gate structure (forget, input, and output gates). This paper has been cited over 13,000 times and established GRU as a standard alternative to LSTM for sequential data.

GRU for Smart Home Applications: Recent research on smart home energy consumption forecasting validates GRU's effectiveness in household time-series prediction. Kim et al. [2] compared SVR, LSTM, GRU, CNN-LSTM, and CNN-GRU models for smart home energy prediction, finding that GRU achieved competitive accuracy while requiring less training time. Similarly, Wang et al. [3] demonstrated that CNN-GRU hybrid models effectively capture temporal patterns in smart grid applications, with GRU's simpler structure enabling faster inference suitable for edge deployment.

Hybrid Approaches: Studies on residential energy forecasting show that hybrid CNN-GRU frameworks outperform standalone LSTM and GRU models for building load prediction, with CNNs extracting spatial features and GRUs providing efficient time-series forecasting. The combination of attention mechanisms with GRU further improves the ability to capture complex temporal patterns in household behavior while maintaining computational efficiency.

5. Multi-Modal Sensor Fusion and Edge AI

Recent Advances in 2024: State-of-the-art multi-modal sensor fusion systems demonstrate remarkable performance improvements through edge computing integration. Recent work shows multimodal fusion algorithms achieving 98.7% sorting accuracy with distributed edge computing architectures enabling 3.2ms average response time through local processing optimization. Edge computing integration with multi-modal perception enables unprecedented capabilities in real-time

adaptation and decision-making, achieving sub-20ms response times for complex fusion operations.

Sensor Fusion Architectures: Research in robotic vision employs various fusion strategies including encoder-decoder frameworks, attention mechanisms, and graph neural networks to process heterogeneous sensory inputs (RGB, Depth, LiDAR, GPS, IMU). These multimodal fusion approaches support core robotic vision tasks such as 3D semantic scene understanding, SLAM, object detection, navigation, and manipulation.

Tactile-Visual Integration: Cutting-edge research demonstrates multimodal tactile sensors utilizing flexible thin-film thermistors for pressure, temperature, texture, and slip detection, achieving ultrasensitive (0.05 mm/s) and ultrafast (4ms) slip sensing for dexterous grasping control. This highlights the trend toward diverse sensor modalities beyond vision and audio.

Market Growth: The multi-modal sensor fusion market is projected to grow from USD 7.1 billion in 2024 to USD 42.5 billion by 2033, reflecting the expanding integration of these technologies across industrial sectors including robotics, autonomous vehicles, healthcare, and smart home systems.

6. Datasets and Benchmarks

IROS 2022 HD10K Dataset: Our visual pollution detection leverages the IROS 2022 dataset [8] specifically designed for household dirt detection, providing 10,000 annotated images of solid waste and liquid stains in residential environments. This domain-specific dataset significantly improves detection accuracy compared to general-purpose object detection datasets.

Ultralytics HomeObjects-3K: This dataset [9] provides 2,689 household object images covering common furniture and appliances, enabling the system to understand room context and spatial semantics beyond simple pollution detection.

Google AudioSet: AudioSet [6] remains the largest-scale collection of manually-annotated audio events, containing over 2 million human-labeled 10-second YouTube clips spanning 632 audio event classes. This dataset forms the foundation for YAMNet's pretraining and enables effective transfer learning to household-specific audio classification.

Connection to Our Work

Our system uniquely combines these established technologies into an integrated on-device AI pipeline:

- **YOLOv11n-pose** provides visual context understanding through object detection and human activity recognition
- **YAMNet** offers auditory context awareness by classifying household environmental sounds
- **FedPer** enables privacy-preserving personalization without transmitting sensitive household data
- **GRU** learns temporal dependencies between multi-modal contexts and zone-specific pollution patterns
- **Multi-modal fusion** at the edge ensures real-time responsiveness without cloud dependency

The novelty lies not in individual components but in their integration into a cohesive, privacy-preserving, context-aware predictive system that transforms reactive robotic cleaning into proactive household automation.

VI. Conclusion

Summary of Achievements

This project successfully developed an AI-driven predictive cleaning system that transforms robotic vacuum cleaners from reactive tools into proactive household partners. By integrating multi-modal sensor fusion (YOLOv11n-pose for visual context, YAMNet for auditory context, and spatial context), the system achieves context-aware pollution prediction rather than relying solely on fixed schedules.

The key technical achievements include:

1. **Multi-modal Context Fusion:** The AttentionContextEncoder successfully combines heterogeneous sensor data (visual, audio, pose, location, temporal) into unified 160-dimensional context vectors with a validation MAE of 0.1005, demonstrating effective cross-modal learning.
2. **Sequential Pollution Prediction:** The GRU-based model learns temporal patterns from 30-timestep sequences and predicts zone-specific cleaning needs with a validation MAE of 0.1313, enabling the system to anticipate contamination before it becomes severe.
3. **Privacy-Preserving Personalization:** By implementing Federated Personalization (FedPer) architecture, the system ensures that sensitive household data never leaves edge devices while still enabling personalized learning through on-device Head Layer fine-tuning.
4. **Real-time Processing Pipeline:** The TimeSyncBuffer algorithm successfully synchronizes asynchronous sensor streams (visual at ~30fps, audio at ~1s intervals) within $\pm 500\text{ms}$ tolerance, enabling consistent real-time inference.

Performance Evaluation

The evaluation results demonstrate that our AI models achieve practical performance levels:

- **YAMNet Audio Classification:** Macro AUC of 0.9899 indicates excellent capability in recognizing household sounds (cooking, dishwashing, vacuum operation, etc.), providing reliable auditory context.
- **YOLO Visual Detection:** mAP50 of 0.566 achieves moderate performance in detecting household objects and pollution (solid waste, liquid stains), sufficient for context-aware decision making.
- **Context Encoder & GRU Prediction:** Combined MAE of approximately 0.10-0.13 demonstrates that the system can reasonably estimate pollution probability, though there is room for improvement in prediction accuracy.

System Significance

This research demonstrates three important contributions to the field of intelligent home robotics:

1. **Privacy by Design:** Unlike cloud-based systems, all AI inference occurs on edge devices (robot vacuum and smartphone tracker), eliminating the transmission of sensitive camera and microphone data to external servers.
2. **Adaptive Personalization:** The FedPer architecture enables household-specific customization without compromising privacy, allowing the robot to learn unique living patterns while sharing only non-identifiable model weights during federated learning.

3. **Proactive Automation:** By predicting when and where cleaning is needed based on multi-modal context, the system reduces manual intervention and improves user experience through intelligent automation.

Limitations

Despite the achievements, several limitations should be acknowledged:

1. **Simulation Environment:** The current implementation uses Webots simulation rather than real-world robot deployment, which may not fully capture the complexity and variability of actual household environments.
2. **Dataset Scale:** The training datasets (~220,000 timesteps for GRU, 12,689 images for YOLO) are generated through simulation rather than real-world collection, potentially limiting model generalization to actual household environments.
3. **Prediction Accuracy:** The GRU model's MAE of 0.131 indicates approximately 13% average error in pollution probability prediction, suggesting that the system may occasionally miss cleaning opportunities or trigger unnecessary cleaning.
4. **Computational Constraints:** While optimized for edge devices, running multiple AI models simultaneously (YOLO, YAMNet, GRU) requires significant computational resources, which may challenge deployment on low-power robot hardware.
5. **Limited Sensor Modalities:** The current system does not incorporate chemical sensors (odor detection) or advanced environmental sensors (humidity, temperature) that could further improve context awareness.

Future Work

Several directions for future research and development include:

1. **Real-world Deployment:** Implementing the system on actual robot vacuum hardware to validate performance under real household conditions, including varying lighting, acoustic noise, and floor types.
2. **Model Improvement:** Exploring advanced architectures such as Transformer-based models for better sequence modeling, and investigating larger-scale datasets to improve prediction accuracy.
3. **Additional Sensor Integration:** Incorporating chemical sensors for odor detection, depth cameras for 3D environment understanding, and contact sensors for physical contamination verification.
4. **Enhanced Federated Learning:** Implementing differential privacy mechanisms to provide mathematical privacy guarantees, and exploring more sophisticated aggregation strategies beyond FedPer.
5. **User Interaction:** Developing intuitive feedback mechanisms that allow users to correct prediction errors, enabling continuous improvement through human-in-the-loop learning.
6. **Energy Optimization:** Implementing model quantization and adaptive inference strategies to reduce power consumption, enabling longer battery life for autonomous operation.
7. **Cross-household Generalization:** Investigating transfer learning techniques to improve initial performance in new households by leveraging patterns learned from other homes while preserving privacy.

This project establishes a foundation for privacy-preserving, context-aware, and personalized robotic cleaning systems. While current performance demonstrates proof-of-concept feasibility, continued research in model accuracy, real-world deployment, and user interaction will be essential for practical commercial adoption.

References

- [1] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling," *arXiv preprint arXiv:1412.3555*, 2014.
 - [2] J. Kim, H. Moon, and B. Kang, "Smart Home Energy Consumption Forecasting Using Deep Learning Models," *Applied Sciences*, vol. 12, no. 5, 2022.
 - [3] Y. Wang, X. Liu, and Z. Chen, "CNN-GRU Hybrid Model for Smart Grid Load Forecasting," *IEEE Transactions on Smart Grid*, 2023.
 - [4] Ultralytics, "YOLOv11: State-of-the-Art Object Detection," *YOLO Vision 2024 Conference*, 2024. Available: <https://docs.ultralytics.com/models/yolo11/>
 - [5] TensorFlow Hub, "YAMNet: Audio Event Classification," Available: <https://tfhub.dev/google/yamnet/1>
 - [6] J. F. Gemmeke, D. P. W. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, "Audio Set: An Ontology and Human-Labeled Dataset for Audio Events," *IEEE ICASSP*, 2017.
 - [7] A. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary, "Federated Learning with Personalization Layers," *arXiv preprint arXiv:1912.00818*, 2019.
 - [8] W. Liu, J. He, and S. Zhang, "HD10K: A Large-Scale Household Dirt Dataset for Robotic Cleaning," *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.
 - [9] Ultralytics, "Home Objects Detection Dataset," Available: <https://docs.ultralytics.com/datasets/detect/>
-